Team047-PT1-Stage 2: Conceptual and Logical Database Design (GradPath)

Explain your assumptions for each entity and relationship in your model. Discuss why you've modeled something as an entity rather than an attribute of another entity. Describe the cardinality of relationships, like why a student is linked to only one advisor. These assumptions might come from customer requirements or application constraints. Please clarify them.

Entity List:

1) USER_PROFILE
2) MAJOR
3) UNIVERSITY
4) JOB_ROLE
5) INDUSTRY
6) PREFERENCE_PRESET
7) LOCATION
8) UNEMPLOYMENT

Entity Details:

USER_PROFILE:

a) User_profile_id (Primary Key): identifier for user profile
b) email: login identifier (each profile has an attached email)
c) grad_year: user graduation year (used for filters)
d) degree_level: highest completed degree level (bachelors, masters, phd)

User_profile is our user entity and represents information the user has to input. It is an important entity for storing user info and is required to produce relevant search queries. Degree_level is stored as an attribute, unlike University and Major, since it does not interact with other parts of the database design and is not supported by our datasets (job_roles do not specify level of degree needed)

MAJOR:

e) major_ID (Primary Key): Identifier for major
f) Major_name: name of major
g) Category: broader grouping for majors (ie. Engineering/Business) -> specifically tied to our datasets

Major represents the majors that users have studied and must enter as baseline info to help filter job roles and analytics. We chose major to be its own entity as opposed to being an attribute of user_profile because our datasets show observed associations of job roles to

majors which would produce useful analytics. Making it an attribute would limit the flexibility of these analytics.

UNIVERSITY:

h) university_ID (Primary Key)
i) university_name: name of University

University represents the institution the user has studied at and is an entity as it is a "place" that is directly tied to the user and exists independently in our university dataset. It is modeled as an entity because multiple users can reference the same institution and the dataset provides structured information about universities.

JOB_ROLE:

j) job_role_ID (Primary Key)
k) job_title: Title of job
l) median_salary: reported median salary

Job_role represents the career outcomes that users may be interested in pursuing. It is modeled as its own entity because job roles are associated with majors based on observed placement data from our datasets, which will help produce filtered analytics. Storing job roles separately prevents repeated job title values and allows flexible association with multiple majors and industries.

INDUSTRY:

m) industry_ID (Primary Key):
n) industry_name

Industry represents broader career sectors that group related job roles. It is modeled as an entity rather than an attribute to allow multiple job roles to belong to the same industry and to enable users to filter job roles by industry through preference presets.

PREFERENCE_PRESET:

o) preset_ID (Primary Key):
p) expected_salary: user entered salary expectation
q) max_unemployment: user entered unemployment filter

Preference_preset represents a saved configuration of user-defined search filters. It allows a user to store multiple preference settings to compare different career scenarios. It is stored as an entity to have user specific configurations persist whenever the user visits the application.

LOCATION:

r) state_ID (Primary Key):
s) state_name

Location represents U.S. states used for filtering and analytics within the application. It is modeled as its own entity to ensure consistent state naming and because unemployment data in our dataset is reported at the state level. Since it will be referenced by presets and unemployment data, possible multiple times per state, it needs to be its own entity.

UNEMPLOYMENT:

t) year (Primary Key that also needs state_ID):
u) unemployment_rate

Unemployment represents state-level unemployment measurements over time that are used in filtering and analytics. It is modeled as its own entity because the dataset provides unemployment rates by state and year. In the relational schema, each unemployment record is uniquely identified by the combination of state_ID and year, which reflects how our dataset is structured.

<u>Relationships and Cardinalities:</u>

1) MAJOR (1) - (0..*) USER_PROFILE

"Each user profile selects exactly one major and many users can share the same major."
Why: Based on datasets we chose to keep the scope to one primary major per user but in the future this may be changed to a many-many relationship to test if users can filter with multiple majors.

2) UNIVERSITY (1) - (0..*) USER_PROFILE

"Each user profile lists exactly one university and many users can attend the same university."
Why: It is possible for a user to have attended multiple universities but based on datasets we have chosen to limit the user to one university.

3) MAJOR (0..*) - (0..*) JOB_ROLE

"A major can be associated with many job roles and a job role can be associated with many majors."
Why: Our datasets do not assert that job roles require specific majors just that suggested ones have an association in the dataset. To reflect that majors are not tied to specific job roles and a job role can have many majors as applicants.

4) INDUSTRY (1) - (0..*) JOB_ROLE

"Each job role belongs to exactly one industry and an industry contains many job roles."
Why: This supports organizing job roles and enables filtering by broader categories.

5) INDUSTRY (0..*) - (0..*) PREFERENCE_PRESET

"A preset can select multiple industries and an industry can be selected by many presets."
Why: Users may want to explore multiple industries per preset.

6) USER_PROFILE (1) - (0..*) PREFERENCE_PRESET

"A user profile can create multiple presets and each preset belongs to exactly one user profile."
Why: Supports saving multiple scenarios/preferences (application constraint).

7) PREFERENCE_PRESET (0..*) - (0..*) LOCATION

"A preset can target multiple states and a state can appear in many presets."
Why: Users may search across several target states.

8) LOCATION (1) - (0..*) UNEMPLOYMENT

"Each state has unemployment measurements across many years. Each unemployment record belongs to one state."
Why: Based on datasets, unemployment is reported by state and year.

Normalize your database. Apply BCNF or 3NF to your schema or show that your schema adheres to one of these normal forms.

<u>Normalized Database:</u>

We choose to normalize the database with 3NF.

USER_PROFILE:

user_profile_ID →grad_year, email, degree_level

Considering foreign keys: user_profile_ID needs major_ID and university_ID for education profile.

user_profile_ID → major_ID,university_ID

This completes 3NF to link functional dependencies.

UNIVERSITY

university_ID → university _name

There is no violation of 3NF in minimal basis as there is no transitive dependency.

MAJOR

major_ID → major_name, category

There is no violation of 3NF in minimal basis.Since job roles have prerequisites of multiple majors and major can link to multiple job roles (many to many) , we need an additional mapping for the relationship for 3NF linking

(major_ID, job_role_ID) → (major_ID,job_role_ID)

This can be achieved by adding a new table MAJOR_JOB_PREREQUISITES. (major_ID [PK, FK], job_role_ID [PK, FK]), so there is a whole key.

JOB_ROLE

job_role_ID → job_name, median_salary

job_role_ID determines a job role on its own without industry_ID so there is no partial dependency. The link to major is already in (major_ID, job_role_ID) → (major_ID,job_role_ID) of MAJOR_JOB_PREREQUISITES. 3NF is completed.

INDUSTRY

industry_ID → industry_name

It does not need foreign keys and the minimal basis satisfies 3NF.

PREFERENCE_PRESET

preset_ID → expected_salary, max_unemployment, user_profile_ID(FK), industry_ID(FK), state_ID(FL). It refers to the industry using join and filtering for queries. The foreign keys only depend on preset_ID so it maintains 3NF.

LOCATION

state_ID → state_name

There is no violation of 3NF in minimal basis

UNEMPLOYMENT

(state_ID,year) → unemployment_rate

This satisfies 3NF with a whole key by adding the foreign dependency.

Convert your conceptual database design (ER/UML) to the logical design (relational schema).

<u>Relational schema:</u>

UNIVERSITY (university_ID: INT [PK], university_name: VARCHAR)

MAJOR (major_ID: INT [PK], major_name: VARCHAR, category: VARCHAR)

INDUSTRY (industry_ID: INT [PK], industry_name: VARCHAR)

LOCATION (state_ID: INT [PK], state_name: VARCHAR)

USER_PROFILE (user_profile_ID: INT [PK], major_ID: INT [FK to MAJOR.major_ID], university_ID: INT [FK to UNIVERSITY.university_ID], grad_year: INT, email: VARCHAR, degree_level: VARCHAR)

JOB_ROLE (job_role_ID: INT [PK], industry_ID: INT [FK to INDUSTRY.industry_ID], job_title: VARCHAR, median_salary: REAL)

UNEMPLOYMENT (year: INT [PK], state_ID: INT [PK, FK to LOCATION.state_ID], unemployment_rate: REAL)

PREFERENCE_PRESET (preset_ID: INT [PK], user_profile_ID: INT [FK to USER_PROFILE.user_profile_ID], industry_ID: INT [FK to INDUSTRY.industry_ID], state_ID: INT [FK to LOCATION.state_ID], expected_salary: REAL, max_unemployment: REAL)

MAJOR_JOB_PREREQUISITES (major_ID: INT [PK, FK to MAJOR.major_ID], job_role_ID: INT [PK, FK to JOB_ROLE.job_role_ID])